COMP 110

Magic Methods + Operator Overloads

Review

What are unique properties of the `__init__` method?
(What sets it apart from other methods?)

Magic Methods

- Methods with built in functionality!
- Not called *directly!*
- Names start and end with two underscores (__<method_name>__)

Question

When I call **`print(x)`**, Python calls *what function* on x *before* printing?

`__str__` Magic Method

- Gives a `str` representation to an object of a Class.
- Call it by calling `str(<class_object>)`

`__str__` Magic Method

- Gives a `str` representation to an object of a Class.
- Call it by calling `str(<class_object>)`

Let's try it together!

Let's define a `__str__` magic method that returns a string of information about a Profile object.

Operator Overloads

- You can write magic methods to give operators meaning!
- Think about operators you use on numbers that you'd like to use on other objects, e.g. +, -, *, /, <, <=, etc…
- This is called operator overloading

`__add__` Magic Method

- Allows you to define addition between an object of the class and another object. (Adds `self` and another object together.)
- Signature: `def __add__(self, <other_object>: <objects_type>)`
- Call it by calling `<class_object> + <other_object>`

Diagram

```python
from __future__ import annotations

class SeedBank:

    num_seeds: int
    plant: str

    def __init__(self, num_seeds: int, plant: str):
        self.num_seeds = num_seeds
        self.plant = plant

    def __str__(self) -> str:
        return f"{self.num_seeds} {self.plant} seeds"

    def __add__(self, more: int) -> SeedBank:
        return SeedBank(self.num_seeds + more, self.plant)

bank1: SeedBank = SeedBank(10, "daisy")
bank2: SeedBank = bank1 + 5
print(bank2)
```

Arithmetic Operator Overloads

+	`__add__(self, other)`
-	`__sub__(self, other)`
*	`__mul__(self, other)`
/	`__truediv__(self, other)`
**	`__pow__(self, other)`
%	`__mod__(self, other)`

Comparison Operator Overloads

`<`	`__lt__(self, other)`
`>`	`__gt__(self, other)`
`<=`	`__le__(self, other)`
`>=`	`__ge__(self, other)`
`==`	`__eq__(self, other)`
`!=`	`__ne__(self, other)`

Class Writing

- Write a class called `ShoppingGuide`
- It should have three attributes, `groceries: list[str]`, `budget: float`, and `store: str`
- Write a *constructor* that takes four parameters:

  `self`, `groceries: list[str]`, `budget: float`, and `store: str`.

  It should update the attributes accordingly.
- Write a `__str__` magic method that gives me all the information of a `ShoppingGuide` object
- Write a magic method `__add__` that takes as parameters self, more_money: float.
  - It should return a new `ShoppingGuide` object with the same attribute values except it should add `more_money` to the budget

Instantiation

- Create a new variable `my_plan` that is reference to a `ShoppingGuide` object with the groceries `["apples", "kiwi"]`, the budget `$5.55`, and the store `"Food Lion"`.
- Now create another variable `AJs_plan` that is my_plan but with $2.12 added to the `budget`
- Print out `AJs_plan`